ПОКРОКОВЕ ЗАВДАННЯ ДЛЯ AI-АГЕНТА

STORKCRYPTO: PRODUCTION FIX, SECURITY І ТРИ ПЛАНИ

Скопіюй цей текст у Google AI Studio.

Роль

Працюй як Senior Full-Stack Engineer, Cloudflare Engineer, Security Engineer, Payment Integration Engineer і Chief Design Developer.

Не обмежуйся локальним preview. Після кожної групи змін перевіряй саме live production:

```
https://storkcrypto.pages.dev/
```

Не став статус `READY`, якщо будь-який P0/P1 acceptance test не пройдено.

ЕТАП 0. Створити baseline і backup

1. Створи окрему git branch:

```
fix/production-stars-ton-plans
```

1. Збережи поточний commit SHA.
2. Збережи список Cloudflare Pages Functions.
3. Збережи список production environment variables без значень секретів.
4. Не виводь секрети у terminal, logs або звіт.
5. Запиши current behavior:

```
```

```
/api/chat
/api/validate
/api/payments/stars/create
/api/payments/stars/status
/tonconnect-manifest.json
```

1. Не видаляй LoadingScreen без backup.
2. Не змінюй 6,5-секундний timing до завершення baseline test.

ЕТАП 1. Видалити production DEV/payment bypass

Знайди і видали з production path:

Plain Text

```
SIMULATE TEST CONFIRMATION (DEV)
tg_user
fake payment
fake transaction
hardcoded isPro=true
```

Правила:

1. DEV simulation може існувати тільки локально.
2. У Cloudflare production ця кнопка не повинна бути у DOM, bundle або response.
3. Не можна активувати PRO через click handler.
4. Не можна активувати PRO через localStorage.
5. Не можна активувати PRO через Zustand state без server confirmation.
6. Wallet connection не може активувати PRO.

Перевір:

Bash

```
grep -R "SIMULATE TEST CONFIRMATION\|tg_user\|fake.*payment\|isPro.*true" .
```

Результат для production:

Plain Text

```
DEV payment bypass: absent
```

ЕТАП 2. Виправити Telegram Stars authentication

2.1. Create invoice endpoint

Endpoint:

```
Plain Text

POST /api/payments/stars/create
```

Обов’язково:

1. Приймати `Authorization: tma <Telegram.WebApp.initData>`.
2. Перевіряти initData HMAC server-side.
3. Отримувати Telegram user id тільки з валідованого initData.
4. Не довіряти user id з JSON body.
5. Не довіряти amount з frontend.
6. Не довіряти currency з frontend.
7. Визначати plan/price на backend.
8. Створювати order у database.
9. Створювати `invoice_payload` з order id.
10. Використовувати тільки `currency: XTR`.

Тести:

```
Plain Text

No auth → 401
Empty initData → 401
Fake initData → 401
Changed user id → 401/403
Valid Telegram initData → 200 + PENDING order + invoiceUrl
```

Не дозволяй:

```
Plain Text

userId: tg_user
```

2.2. Відкриття invoice

Frontend flow:

```
Plain Text

validate initData
→ create Stars invoice
→ Telegram.WebApp.openInvoice(invoiceUrl)
→ callback only triggers refresh
→ backend status polling
```

Якщо `Telegram.WebApp.initData` відсутній:

```
Plain Text

Показати: Відкрийте застосунок усередині Telegram
Не створювати order
Не показувати success
```

ЕТАП 3. Реалізувати повний payment webhook

3.1. pre_checkout_query

Webhook повинен:

1. Знайти order по payload.
2. Перевірити user id.
3. Перевірити `currency === XTR`.
4. Перевірити amount.
5. Перевірити plan.

6. Перевірити `PENDING` status.
7. Відповісти Telegram протягом 10 секунд.
8. Відхилити invalid order.

3.2. successful_payment

PRO activation дозволена тільки після:

```
Plain Text

message.successful_payment
```

Зберегти server-side:

```
Plain Text

telegram_payment_charge_id
provider_payment_charge_id, якщо є
telegram_user_id
invoice_payload
currency
amount
plan
paid_at
```

У database додати unique constraint на:

```
Plain Text

telegram_payment_charge_id
```

Зробити webhook idempotent.

Повторний webhook:

```
Plain Text

не створює другу payment record
не продовжує підписку вдруге
не дублює access
```

ЕТАП 4. Реальний status API

Створити:

```
Plain Text

GET /api/payments/stars/status?orderId=...
```

Вимоги:

```
Plain Text

No auth → 401 JSON
Unknown order → 404 JSON
Foreign order → 403 JSON
Valid pending → 200 JSON PENDING
Valid paid → 200 JSON PAID
```

Ніколи не повертати `index.html` з `/api/*`.

Приклад:

```
JSON

{
  "orderId": "...",
  "status": "PAID",
  "paymentVerified": true,
  "subscriptionActive": true,
  "planId": "PRO",
  "expiresAt": "..."
}
```

Frontend не має права показувати `PRO ACTIVE` до `paymentVerified === true`.

ЕТАП 5. Реалізувати три платні плани

Зберегти окремий безкоштовний `BASIC` і створити три paid tiers.

План 1: STARTER

```
Plain Text
```

```
planId: STARTER
price: 99 XTR
duration: 30 days
```

Функції:

- актуальні market signals з fair-use;
- 10–20 AI scans на день;
- базові alerts;
- Basic Academy;
- базовий Whale Radar.

План 2: PRO

Plain Text

```
planId: PRO
price: 250 XTR
popular: true
duration: 30 days
```

Функції:

- Unlimited Signals з fair-use limit;
- AI Market Scan;
- Whale Radar;
- Liquidation Heatmap;
- no ads;
- повна Academy;
- advanced alerts;
- dashboard customization.

План 3: WHALE

Plain Text

```
planId: WHALE
price: 500 XTR
```

```
duration: 30 days
```

Функції:

- усе з PRO;
- VIP Whale Radar;
- advanced on-chain intelligence;
- custom alerts;
- priority AI queue;
- early access;
- розширені metrics.

Правила:

1. Ціни зберігати на backend.
2. Frontend передає тільки `planId`.
3. Не дозволяти frontend передавати arbitrary amount.
4. Усі плани оплачуються Stars `XTR`.
5. Не додавати GRAM, TON або MetaMask payment alternatives.
6. У PRO modal показати всі три плани.
7. Додати `Popular` лише до PRO.
8. Для кожного плану показати чіткі переваги.
9. Backend перевіряє, що planId дозволений.
10. Webhook активує саме обраний planId.

ЕТАП 6. TON Wallet

1. Залишити тільки TON Connect.
2. Видалити MetaMask та exchange wallet options.
3. Додати один root `TonConnectUIProvider`.
4. Не монтувати його у кожній вкладці.
5. Додати Portfolio button:

```
Plain Text
```

```
Connect TON Wallet
```

1. Після connect picker автоматично закривається.
2. Показати:

```
Plain Text

TON Wallet connected
UQ12…ABCD
Copy
Explorer
Disconnect
```

1. Використовувати `useTonAddress`, `useTonWallet`, `useIsConnectionRestored`.
2. Не вважати wallet connection payment.
3. Не зберігати seed/private key.

Manifest:

```
JSON

{
  "url": "https://storkcrypto.pages.dev",
  "name": "StorkCrypto",
  "iconUrl": "https://storkcrypto.pages.dev/icon-180.png",
  "termsOfUseUrl": "https://storkcrypto.pages.dev/terms",
  "privacyPolicyUrl": "https://storkcrypto.pages.dev/privacy"
}
```

Перевірити:

```
Plain Text

GET /tonconnect-manifest.json → 200 JSON
GET /icon-180.png → 200 image/png
```

ЕТАП 7. Виправити Academy localization

Кнопки UA/EN/RU повинні змінювати весь content.

Для кожної мови перекласти:

- Academy title;
- card title;
- description;
- level;
- tags;
- CTA;
- lesson title;
- video title;
- quiz;
- errors;
- paywall;
- loading/fallback.

Acceptance:

```
UA → український content
EN → English content
RU → русский content
```

Зміна лише `aria-pressed` — FAIL.

ЕТАП 8. Market data correctness

Уніфікувати статуси:

```
LIVE
DELAYED
STALE
DEMO
ERROR
```

Rules:

- LIVE тільки після свіжого API response;

- STALE тільки для cache після threshold;
- DEMO тільки для simulator;
- ERROR при відмові API;
- ticker, summary і charts використовують однаковий статус;
- показувати `lastUpdatedAt`;
- показувати Retry;
- не писати LIVE для cached data.

ЕТАП 9. Loading screen і дизайн

Не замінювати Manifesto.

Зберегти:

- vertical branded composition;
- dark background;
- 6,5 секунд;
- один рядок `StorkCrypto` на 360×800;
- no blank flash;
- no semicircle disappearance;
- no `GENESIS`.

Бажана послідовність:

```
rotating concentric rings
→ fog
→ StorkCrypto emergence
→ fracture
→ clear pixels
→ controlled pixel fall
→ horizontal data convergence
→ dashboard
```

Після loading onboarding повинен бути окремим і не виглядати як зависання.

Стиль:

- dark;

- professional;
- futuristic;
- readable;
- non-multicolor;
- compact logo;
- controlled accent.

ЕТАП 10. Modals

Правило:

```
Plain Text

Informational/technical modal → fullscreen
PRO subscription modal → compact exception
```

Перевірити:

- z-index;
- safe area;
- `role=dialog` ;
- `aria-modal=true` ;
- close;
- Escape;
- scroll lock;
- cleanup;
- no orphan overlay.

Зробити 30 циклів для:

- asset details;
- Neural Summary;
- Academy;
- News;
- Portfolio;

- PRO;
- Wallet;
- Error/retry.

ЕТАП 11. Security scan

Перевірити:

```
bot token y frontend → absent
Supabase service role y frontend → absent
Gemini secret y frontend → absent
admin credentials → absent
private keys → absent
seed phrases → absent
tg_user → absent
DEV payment button → absent
fake payment success → absent
```

Додати:

- rate limits;
- request body validation;
- idempotency;
- CSP;
- secure headers;
- no open redirect;
- safe external links;
- no sensitive initData logging.

ЕТАП 12. Cloudflare deployment

Після commit:

1. Push branch.
2. Перевір Cloudflare build.

3. Перевір deployment SHA.
4. Перевір Functions deployment.
5. Перевір secrets names, не показуючи values.
6. Перевір `/api/*` на live URL.
7. Перевір cache behavior.
8. Зроби production smoke test.
9. Очисти старий deployment/cache, якщо потрібно.
10. Зафіксуй live URL і timestamp тесту.

ЕТАП 13. Test transaction protocol

Не списуй реальні кошти без окремого підтвердження користувача.

Спочатку виконати safe tests:

Plain Text

```
1. No auth create → 401.
2. Fake auth create → 401.
3. Valid Telegram context → invoiceUrl.
4. Invoice opens in Telegram.
5. Cancel → BASIC.
6. Payment pending → not PRO.
7. Backend receives pre_checkout_query.
8. Backend receives successful_payment.
9. Backend verifies charge id.
10. PRO activates only after verification.
11. Duplicate webhook is idempotent.
12. Expired order cannot activate PRO.
```

Якщо потрібна реальна тестова купівля, спочатку покажи користувачу:

Plain Text

```
plan
price in XTR
duration
what will happen
whether real Stars will be spent
```

і дочекайся окремого підтвердження.

ЕТАП 14. Фінальний самоконтроль

Виконай:

Plain Text

```
AI Chat 10 requests
Stars auth tests
Stars invoice open test
successful_payment test
duplicate webhook test
three plans test
TON connect/disconnect 30 cycles
modal open/close 30 cycles
Academy UA/EN/RU
Day/Night
loading timing
mobile 360x800
mobile 390x844
console errors
network errors
production bundle secrets scan
Cloudflare live verification
```

Формат фінального звіту

Plain Text

```
1. Commit SHA:
2. Cloudflare deployment SHA:
3. Змінені файли:
4. API results:
5. Stars results:
6. Successful payment result:
7. Three plans result:
8. TON Wallet result:
9. Academy result:
10. Market status result:
11. Loading result:
12. Modal stress test result:
13. Security scan result:
```

```
14. Залишкові ризики:
15. READY / NOT READY:
```

Не став `READY`, якщо:

Plain Text

```
DEV simulation є у production
successful_payment не перевірений
PRO активується без server confirmation
один із payment API повертає HTML
wallet picker не закривається
Academy не змінює content
orphan overlays > 0
Cloudflare deployment не підтверджений
```